

Вариант 3: RGB ↔ XYZ ↔ LAB ↔ CMYK

Цель работы

Целью работы является изучение цветовых моделей **RGB**, **XYZ**, **CIE LAB** и **CMYK** и алгоритмов перехода между ними, а также разработка **интерактивного веб-приложения**, отображающего цвет одновременно в четырёх моделях с возможностью редактирования из любой из них. Все преобразования должны быть реализованы вручную, без сторонних библиотек, а матрицы перехода — не задаваться константами, а вычисляться программно. Приложение должно поддерживать выбор стандарта освещения, выбор алгоритма цветоделения CMYK, обработку выхода цвета за цветовой охват с предупреждением пользователя, а также отображать на ползунках динамические градиенты.

Ход работы

В ходе работы были **изучены** цветовые модели **RGB**, **XYZ**, **LAB** и **CMYK**, их компоненты, диапазоны значений и алгоритмы перехода между ними. Переход **RGB** ↔ **XYZ** выполняется с помощью матрицы, переход **XYZ** ↔ **LAB** — по формулам **CIE 1976** относительно опорного белого, переход **RGB** ↔ **CMYK** — по формулам цветоделения. Таким образом, **LAB** связан с **RGB** через промежуточную модель **XYZ**, а **CMYK** вычисляется непосредственно из **RGB**.

Матрицы перехода не заданы жёстко, а вычисляются программно из определений стандартов: цветностей основных цветов sRGB — $r(0.64; 0.33)$, $g(0.30; 0.60)$, $b(0.15; 0.06)$ — и цветности белой точки. В интерфейсе выбирается стандарт освещения **D65**, **D50** или **E** с цветностями $(0.3127; 0.3290)$, $(0.3457; 0.3585)$ и $(1/3; 1/3)$ соответственно; в **XYZ** их белые точки равны $(0.95046; 1; 1.08906)$, $(0.96430; 1; 0.82510)$ и $(1; 1; 1)$. Каждая цветность переводится в **XYZ** по формуле $(x/y; 1; (1 - x - y)/y)$. Из координат основных цветов составляется матрица P , находятся масштабные коэффициенты $S = P^{-1} \cdot W$, и итоговая матрица получается как $M = P \cdot \text{diag}(S)$. Для D65 она совпадает со стандартной матрицей sRGB $(0.4124, 0.3576, 0.1805 \dots)$.

Поскольку стандарт sRGB определён для белого D65, для другого источника применяется **хроматическая адаптация Брэдфорда**: $M_{\text{adapt}} = B^{-1} \cdot \text{diag}(B \cdot W_{\text{dst}} / B \cdot W_{\text{src}}) \cdot B$, а итоговая матрица равна $M_{\text{adapt}} \cdot M_{\text{sRGB}}$. Обратная матрица M^{-1} вычисляется через алгебраические дополнения. При смене источника матрица и опорный белый для **LAB** пересчитываются на лету: видимый цвет сохраняется, а его координаты **XYZ** и **LAB** меняются. Текущая матрица отображается в интерфейсе.

Перед умножением на матрицу компоненты **RGB** линеаризуются по **гамма-кривой sRGB**: $C_{\text{lin}} = C / 12.92$ при $C \leq 0.04045$, иначе $C_{\text{lin}} = ((C + 0.055) / 1.055)^{2.4}$. Обратное преобразование: $C = 12.92 \cdot C_{\text{lin}}$ при $C_{\text{lin}} \leq 0.0031308$, иначе $C = 1.055 \cdot C_{\text{lin}}^{(1/2.4)} - 0.055$. Координаты **LAB** вычисляются относительно белой точки выбранного источника (X_n, Y_n, Z_n) с константами $\epsilon = 216/24389$ и $\kappa = 24389/27$: $L = 116 \cdot f(Y/Y_n) - 16$, $a = 500 \cdot (f(X/X_n) - f(Y/Y_n))$, $b = 200 \cdot (f(Y/Y_n) - f(Z/Z_n))$, где $f(t) = \sqrt[3]{t}$ при $t > \epsilon$ и $f(t) = (\kappa \cdot t + 16)/116$ в противном случае. Обратный переход выполняется через $f_y = (L + 16)/116$, $f_x = f_y + a/500$, $f_z = f_y - b/200$, после чего $x_r = f_x^3$ при $f_x^3 > \epsilon$, иначе $(116 \cdot f_x - 16)/\kappa$; $y_r = f_y^3$ при $L > \kappa\epsilon$, иначе L/κ ; z_r вычисляется аналогично x_r .

Для перехода в **CMYK** сначала вычисляются $C = 1 - R/255$, $M = 1 - G/255$, $Y = 1 - B/255$ и серая компонента $g = \min(C, M, Y)$. Реализованы два алгоритма цветоделения. При **GCR** (замена серой компоненты) $K = g$, то есть серая составляющая заменяется чёрным во всём

диапазоне. При **UCR** (удаление подложечного цвета) чёрный вводится только в глубоких тенях с порогом $t = 0.5$: $K = 0$ при $g \leq t$, иначе $K = g \cdot (g - t) / (1 - t)$. В светлых тонах $K = 0$, а к чёрному значение плавно растёт до полной замены. Затем цветные компоненты нормируются: $C' = (C - K) / (1 - K)$, аналогично M и Y . Обратный переход одинаков для обоих алгоритмов: $R = 255 \cdot (1 - C') \cdot (1 - K)$. Например, $RGB(51, 51, 51)$ при GCR даёт $CMYK(0, 0, 0, 80 \%)$, а при UCR — $CMYK(61.5, 61.5, 61.5, 48 \%)$. Значения $CMYK$ можно редактировать напрямую.

Для обработки некорректных цветов реализована **проверка выхода за цветовой охват**. После преобразования $XYZ \rightarrow$ линейный RGB проверяется, попадают ли гамма-кодированные значения в диапазон $[0, 1]$ с допуском в пол-кода ($0.5/255$). Если нет, пользователю выводится предупреждение и применяется выбранная стратегия. При **Clipping** каждое значение ограничивается соответствующей границей: $v' = \text{clamp}(v, 0, 1)$. При **Scaling** в линейном пространстве диапазон $[lo, hi]$, где $lo = \min(0, R, G, B)$ и $hi = \max(1, R, G, B)$, пропорционально сжимается в $[0, 1]$ по формуле $v' = (v - lo) / (hi - lo)$, благодаря чему соотношения между компонентами сохраняются. Например, $LAB(50, 100, -100)$ при Clipping даёт $RGB(180, 0, 255)$, а при Scaling — $RGB(156, 0, 255)$.

Практическая часть заключалась в разработке веб-приложения с четырьмя основными блоками: **RGB, XYZ, LAB и CMYK**. Приложение позволяет задавать цвет тремя способами: **ползунками, числовыми полями и палитрой**. Дополнительно отображается предпросмотр текущего цвета и его значение в формате **HEX**. При изменении любой координаты значения в остальных моделях автоматически пересчитываются.

На ползунках реализованы **динамические градиенты**: фон каждого ползунка показывает, какой цвет получится при его перемещении при текущих значениях остальных компонентов, во всех моделях. Для этого значение компонента перебирается от минимума до максимума, и каждый вариант переводится в RGB . Для RGB и $CMYK$ достаточно двух точек, так как зависимость линейная, а для XYZ и LAB используется 16 точек. Полученные цвета задаются как `linear-gradient` в фоне ползунка.

Структура приложения разделена по принципу **Model–View–ViewModel (MVVM)**. **Model** (`colorMath.js`) содержит чистую математику преобразований и ничего не знает об интерфейсе. **ViewModel** (`colorViewModel.js`) хранит состояние, выполняет пересчёт и строит градиенты, не обращаясь к `DOM`. **View** (`colorView.js`) отвечает за разметку, ползунки и обработку событий, которые превращаются в вызовы команд `ViewModel`: `setComponent`, `setIlluminant`, `setGamutMode`, `setCmykMode` и `setHex`. Модуль `main.js` связывает `View` и `ViewModel`: `View` подписан на состояние `ViewModel` и перерисовывается при каждом его изменении.

Корректность работы проверена **автоматическими тестами** (25 тестов) для `Model` и `ViewModel`. Тесты математики проверяют, в частности, что $RGB(255, 0, 0)$ при D65 даёт $XYZ(0.4124, 0.2126, 0.0193)$ и $LAB(53.24, 80.09, 67.20)$, а при D50 — $LAB(54.29, 80.80, 69.89)$; что $RGB(0, 255, 0)$ при D65 переводится в $LAB(87.73, -86.18, 83.18)$, а $RGB(0, 0, 255)$ — в $LAB(32.30, 79.19, -107.86)$; что $RGB(255, 0, 0)$ при GCR и UCR даёт $CMYK(0, 100, 100, 0)$, а $RGB(255, 255, 255)$ при любом источнике — $LAB(100, 0, 0)$. Также проверяется, что построенная матрица D65 совпадает со стандартной, что $M \cdot M^{-1} = I$, что выполняется обратимость $RGB \rightarrow LAB \rightarrow RGB$ и $RGB \rightarrow CMYK \rightarrow RGB$, а также поведение UCR, GCR, Clipping и Scaling. Тесты `ViewModel` проверяют пересчёт при смене источника, стратегии охвата, алгоритма цветоделения и построение градиентов ползунков.

Инструментарий

При разработке использовались HTML5, CSS3 и JavaScript (ES-модули) без сторонних библиотек. HTML применялся для создания структуры интерфейса, CSS — для

оформления элементов приложения, JavaScript — для реализации преобразований цветовых моделей, автоматического пересчёта и взаимодействия между Model, View и ViewModel. Для автоматических тестов использовался встроенный в Node.js модуль node:test (Node 18+). Приложение размещено на GitHub Pages и работает в браузере Chrome. GitHub использовался для хранения исходного кода проекта и контроля его версий.

Выводы

В ходе лабораторной работы были изучены цветовые модели RGB, XYZ, CIE LAB и CMYK и реализованы преобразования между ними. Были рассмотрены построение матриц перехода, хроматическая адаптация, гамма-коррекция sRGB, алгоритмы цветоделения и способы обработки цветов, выходящих за цветовой охват.

Создано интерактивное веб-приложение, позволяющее задавать цвет ползунками, числовыми полями и палитрой с автоматическим пересчётом во всех четырёх моделях. Матрицы перехода не заданы константами, а вычисляются из цветностей основных цветов и выбранного источника (D65, D50, E) с адаптацией Брэдфорда. Реализованы два алгоритма цветоделения CMYK (GCR и UCR), две стратегии обработки выхода за охват (Clipping и Scaling) с предупреждением пользователя и динамические градиенты на ползунках.

Разделение приложения на Model, View и ViewModel позволило отделить математические преобразования от пользовательского интерфейса и покрыть их автоматическими тестами, которые подтвердили корректность вычислений. Таким образом, основные требования технического задания выполнены.